P2249R3
Mixed comparisons for smart pointers

Published Proposal, 2021-11-10

Issue Tracking:
Inline In Spec

Author:
Giuseppe D'Angelo

Audience:
LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

We propose to enable mixed comparisons for the Standard Library smart pointer class templates `unique_ptr` and `shared_ptr`, so that one can compare them against raw pointers.

Table of Contents

1. Changelog

- R3
  - Clarified some design decisions after a review on the LEWG mailing list.
  - Changed the proposed wording for the constraints, in order to exclude types that are derived from `unique_ptr` or `shared_ptr` specializations.
- R2
  - Fixes to the proposed wording.
- R1
  - Added some clarifications to the design decisions, following a review on the LEWG mailing list.
  - Made the proposed operators hidden friends.
  - Rebased on top of the latest draft of the Standard.
  - Minor fixes.
- R0
  - First submission.

§ 2. Tony Tables

Before

```
class Manager
{
    // The Manager owns the Objects, therefore it uses smart pointers.
    // The Manager gives out non-owning raw pointers to clients.
    // A client gives the raw pointer back to the Manager, to tell it
    // to act on that Object; the Manager has then to look up the Object
    // in its storage.

    std::vector<std::unique_ptr<Object>> objects;

public:
    Object* get_object(~~~) const
    {
        return objects[~~~].get();
    }

    void drop_object(Object* input)
    {
        // Must use erase_if and a custom comparison (e.g. a lambda).
        auto isEqual = [input](const std::unique_ptr<Object>& o) {
            return o.get() == input;
        };
        erase_if(objects, input);
    }

    ssize_t index_for_object(Object* input) const
    {
        // Same story.
        // Code like this (predicates, etc.) may get duplicated all over the place
        // where smart pointers are used in containers/algorithms. Surely,
        // centralizing it is good practice, but there's always the temptation of
        // just writing the one-liner lambda and "moving on" rather than refactoring...
        auto isEqual = [input](const std::unique_ptr<Object>& o) {
            return o.get() == input;
        };
        auto it = std::ranges::find_if(objects, isEqual);
        // etc.
    }
};
```

```
class Manager
{
    std::vect

public:
    Object* g
    {
        retur
    }

    void drop
    {
        // Ju
        erase
    }

    ssize_t i
    {
        // Sa
        // Un
        // st
        // is
        auto
        // et
    }
};
```

```
// Suppose insteat that the Manager needs to use an associative container rather than
// a sequential container (e.g. mapping some data to each object).
// Then, an heterogeneous comparator becomes a necessity -- we can't possibly
// look up a unique_ptr using another unique_ptr to the same object, especially
// if clients give us non-owning raw pointers to act upon.

// Heterogeneous comparator
template <class T> struct smart_pointer_comparator {
    struct is_transparent {};

    bool operator()(const std::unique_ptr<T>& lhs, const std::unique_ptr<T>& rhs) const
    { return lhs < rhs; }
    bool operator()(const std::unique_ptr<T>& lhs, const T* rhs) const
    { return std::less()(lhs.get(), rhs); }
    bool operator()(const T* lhs, const std::unique_ptr<T>& rhs) const
    { return std::less()(lhs, rhs.get()); }
};

// A sorted associative container with some data
std::map<std::unique_ptr<Object>, Data,
```

```
// No need fo

// ... just u
std::map<std:
```

Before	

`    smart_pointer_comparator<Object>> objects = ~~~;`
 	
`// Heterogeneous lookup using a raw pointer`
`object* ptr = ~~~;`
`auto it = objects.find(ptr);`
`if (it != objects.end()) { use(it->second); }`
	`    std:less<`
	
	`// Heterogene`
	`Object* ptr =`
	`auto it = obj`
	`if (it != obj`

`// Same, with an unordered associative container.`
 	
`// Heterogeneous hasher; [util.smartptr.hash] guarantees that both the`
`// specializations below return the very same value for the same pointer.`
`template <class T> struct smart_pointer_hasher`
`{`
`    struct is_transparent {};`
`    size_t operator()(const std::unique_ptr<T>& ptr) const {`
`        // equal by definition to std::hash<T*>(ptr.get()), that is, (*this)(ptr.get())`
`        return std::hash<std::unique_ptr<T>>()(ptr);`
`    }`
`    size_t operator()(T* ptr) const {`
`        return std::hash<T*>()(ptr);`
`    }`
`};`
 	
`// Heterogeneous equality comparator`
`template <class T> struct smart_pointer_equal`
`{`
`    struct is_transparent {};`
`    bool operator()(const std::unique_ptr<T>& lhs, const std::unique_ptr<T>& rhs) const`
`    { return lhs == rhs; }`
`    bool operator()(const std::unique_ptr<T>& lhs, const T* rhs) const`
`    { return lhs.get() == rhs; }`
`    bool operator()(const T* lhs, const std::unique_ptr<T>& rhs) const`
`    { return lhs == rhs.get(); }`
`};`
 	
`std::unordered_map<std::unique_ptr<Object>, Data,`
`    smart_pointer_hasher<Object>,`
`    smart_pointer_equal<Object>> objects = ~~~;`
 	
`// Heterogeneous lookup`
`Object* ptr = ~~~;`
`auto it = objects.find(ptr);`
`if (it != objects.end()) { use(it->second); }`
	`// [P0919R3]`
	`// so a custo`
	
	`template <cla`
	`{`
	`    struct is`
	`    size_t op`
	`        // eq`
	`        retur`
	`    }`
	`    size_t op`
	`        retur`
	`    }`
	`};`
	
	`// Custom het`
	
	`std::unordere`
	`    smart_poi`
	`    std::equa`
	
	`// Heterogene`
	`Object* ptr =`
	`auto it = obj`
	`if (it != obj`

§ 3. Motivation and Scope

Smart pointer classes are universally recognized as the idiomatic way to express ownership of a resource (very incomplete list: [Sutter], [Meyers], [R.20]). On the other hand, raw pointers (and references) are supposed to be used as non-owning types to access a resource.

Both smart pointers and raw pointers, as their name says, share a common semantic: **representing the address of an object**.

This semantic comes with a set of meaningful operations; for instance, asking if two (smart) pointers represent the address of the same object. `operator==` is used to express this intent.

Indeed, with the owning smart pointer class templates available in the Standard Library (`unique_ptr` and `shared_ptr`), one can already use `operator==` between two smart pointer objects (of the same class). However one *cannot* use it

between a smart pointer and a raw pointer, because the Standard Library is lacking that set of overloads; instead, one has to manually extract the raw pointer out of the smart pointer class:

```
std::shared_ptr<object> sptr1, sptr2;
object* rawptr;

// Do both pointers refer to the same object?
if (sptr1 == sptr2) { ~~~ }        // WORKS
if (sptr1 == rawptr) { ~~~ }       // ERROR, no such operator
if (sptr1.get() == rawptr) { ~~~ } // WORKS; but why the extra syntax?
```

This discussion can be easily generalized to the full set of the six relational operations; these operations have already well-established semantics, and are indeed already defined between smart pointers objects (of the same class) or between raw pointers, but they are not supported in mixed scenarios.

> We propose to remove this inconsistency by defining the relational operators between the Standard Library owning smart pointer classes and raw pointers.

Allowing mixed comparisons isn't merely a "semantic fixup"; the situation where one has to compare smart pointers and raw pointers commonly occurs in practice (the typical use case is outlined in the first example in the § 2 Tony Tables above, where a "manager" object gives non-owning raw pointers to clients, and the clients pass these raw pointers back to the manager, and now the manager needs to do mixed comparisons).

§ 3.1. Associative containers

Moreover, we believe that allowing mixed comparisons is useful in order to streamline heterogeneous comparison in associative containers for smart pointer classes.

The case of an associative container using a `unique_ptr` as its key type is particularly annoying; one cannot practically *ever* look up in such a container using another `unique_ptr`, as that would imply having two `unique_ptr` objects owning the same object. Instead, the typical lookup is heterogeneous (by raw pointer); this proposal is one step towards making it more convenient to use, because it enables the usage of the standard `std::less` or `std::equal_to`.

We however are not addresssing at all the issue of heterogeneous hashing for smart pointers. While likely very useful in general, heterogeneous hashing can be tackled separately by another proposal that builds on top of this one (for instance, by making the `std::hash` specializations for Standard smart pointers 1) *transparent*, and 2) able to hash the smart pointer's `pointer_type` / `element_type*` as well as the smart pointer object itself. But more research and field experience is certainly needed.)

§ 4. Impact On The Standard

This proposal is a pure library extension. It proposes changes to an existing header, `<memory>`, but it does not require changes to any standard classes or functions and it does not require changes to any of the standard requirement tables. The impact is positive: code that was ill-formed before becomes well-formed.

This proposal does not depend on any other library extensions.

This proposal does not require any changes in the core language.

[P0805R2] is vaguely related to this proposal. It proposes to add mixed comparisons between containers of the same type (for instance, to be able to compare a `vector<int>` with a `vector<long>`), without resorting to manual calls to algorithms; instead, one can use a comparison operator. A quite verbose call to `std::equal(v1.cbegin(), v1.cend(), v2.cbegin(), v2.cend())` can therefore be replaced by a much simpler `v1 == v2`. In this sense, [P0805R2] matches the spirit of the current proposal, although comparing smart pointers and raw pointer does not require any algorithm, and does not have such a verbose syntax.

5. Design Decisions

5.1. Should `unique_ptr` have the full set of ordering operators (`<`, `<=`, `>`, `>=`), or just `<=>`?

[P1614R2] added support for `operator<=>` across the Standard Library. Notably, it *added* `operator<=>` for `unique_ptr`, leaving the other four ordering operators (`<`, `<=`, `>`, `>=`) untouched. On the other hand, when looking at `shared_ptr`, the same paper *replaced* these four operators with `operator<=>`.

We believe that this was done in order to preserve the semantics for the existing operators, which are defined in terms of customization points (notably, `common_type`; `unique_ptr` can work in terms of a custom "fancy" pointer type). We are not bound by any pre-existing semantics, so we are just proposing `operator<=>` for `unique_ptr`.

> ISSUE 1 What does LEWG(I) think about this?

5.2. Should operators for a `smart_pointer<T>` accept precisely `T*` or anything convertible to `T*`?

Technically speaking, neither: we are proposing to accept anything which is *comparable* to `T*`.

The rationale of this proposal is that smart pointers should act like raw pointers when it comes to comparison operators. For instance, raw pointers allow for mixed comparisons if both pointers are convertible to their composite pointer type ([expr.type]):

```
Base* b = ~~~;
Derived* d = ~~~;

if (b == d) { ~~~ } // OK
```

Therefore, we want the following to also work:

```
std::unique_ptr<Base> b = ~~~;
Derived* d = ~~~;

if (b == d) { ~~~ } // OK, with this proposal
```

As mentioned, the design that we are proposing is actually more general than simply accepting `T*` or types convertible to `T*`: we are proposing to accept any data type that can be compared against `T*`. This may include, for instance, non-owning/observer pointers like `observer_ptr` (not in the Standard). Such pointer classes are not necessarily implicitly convertible to a raw pointer type. Yet, it makes completely sense to allow for a mixed comparison against them (provided that they also define mixed comparison operators). For instance:

```
std::unique_ptr<Object> owning_ptr = ~~~;

// non-owning; not in the Standard
observer_ptr<Object> non_owning_ptr(owning_ptr);

if (owning_ptr == non_owning_ptr) { ~~~ }
```

The last comparison makes semantic sense, and is therefore allowed by this proposal.

Please note that the existing comparison operators for smart pointers already allow for mixed comparisons (between different specializations of the same smart pointer class template). This proposal is therefore not introducing any inconsistency.

5.3. Would these operations make the `equality_comparable_with` or `three_way_comparable_with` concepts satisfied between a smart pointer and a raw pointer?

No. Adding the operations would indeed bring the Standard Library's smart pointer classes "one step closer" to satisfy those concepts, but they would still be unsatisfied because there is no `common_reference_t` between a smart pointer and a raw

pointer.

Changing that is out of scope for the present proposal (and orthogonal to it).

> In general, the current situation of comparison concepts and Standard Library smart pointers is "suboptimal". For instance:
>
> ```
> static_assert(equality_comparable_with<unique_ptr<int>, nullptr_t>);  // ERROR (1)
> static_assert(equality_comparable_with<shared_ptr<int>, nullptr_t>);  // OK    (2)
>
> static_assert(three_way_comparable_with<unique_ptr<int>, nullptr_t>); // ERROR (3)
> static_assert(three_way_comparable_with<shared_ptr<int>, nullptr_t>); // ERROR (4)
> ```
>
> ... despite the existence of the related operators between smart pointers and `std::nullptr_t`. (1) and (3) fail because eventually the concepts are going to require `unique_ptr` to be copiable. (3) and (4) fail because they require `std::nullptr_t` to be three-way comparable, which it isn't (`std::nullptr_t` lacks relational operations).
>
> An analysis and discussion is available in this thread on StackOverflow. [P2403] and [P2404] are aiming at closing these semantics gaps (for `std::nullptr_t`, not for raw pointers in general).

An unfortunate consequence of this is that, for the moment being, we will yet not be able to use many range-based algorithms using heterogeneous comparisons:

```
std::vector<std::unique_ptr<Object>> objects;

Object *ptr = ~~~;

auto it = std::ranges::find(objects, ptr); // ERROR; must use find_if and a predicate
```

On the other hand, non-range algorithms would work just fine:

```
auto it = std::find(objects.begin(), objects.end(), ptr); // OK
erase(objects, ptr); // OK
```

§ 5.4. Should mixed comparisons between *different* smart pointer classes be allowed?

There are some considerations that in our opinion apply to this case.

The first is whether such an operation makes sense. From the abstract point of view of "comparing the addresses of two objects", the operation is meaningful, even if the addresses are represented by instances of different smart pointer classes. As mentioned before, the currently existing comparison operators of the smart pointer classes implement these semantics.

> Technically speaking, they implement a *superset* of these semantics, as they use `std::less` or `std::compare_three_way` and therefore always yield a strict weak ordering, even when the built-in comparison operator for pointers would not guarantee any ordering.
>
> For instance:
>
> ```
> std::unique_ptr<int> a(new int(123));
> std::unique_ptr<int> b(new int(456));
>
> if (a < b)             { ~~~ } // well-defined behavior
> if (a.get() < b.get()) { ~~~ } // unspecified behavior [expr.rel/4.3]
> ```
>
> The operations that we are proposing would also similarly work via `std::compare_three_way`.

On the other hand: the Standard Library smart pointer classes that this proposal deals with are all *owning* classes. One could therefore reason that there is little utility at allowing a mixed comparison, because it's likely to be a mistake by the user. In a

"ordinary" program, such comparisons would inevitably yield false, because the same object cannot be owned by two smart pointers of different classes (if it is, there is a bug in the program).

There is some semantic leeway here, represented by the fact that the smart pointer classes can hold custom deleters (incl. empty/no-op deleters), aliased pointers (in the case of `shared_ptr`), as well as as fancy pointer types (in the case of `unique_ptr`). In principle, one can write a perfectly legal example where the same object is owned by smart pointers of different classes, using custom deleters and/or custom fancy pointer types, and then wants to compare the smart pointers (compare the addresses of the objects owned by them).

In some ways, this is hardly a realistic use case for allowing comparisons between different smart pointer classes. The danger of misuse of such comparisons, again in "ordinary" code, seems be stricly greater than their usefulness, given the unlikelihood of valid use cases -- in the majority of ordinary usages, the comparisons would be meaningless.

We have therefore a tension between the abstract/ideal domain of the operations, and the practical usage. The problem is that **trying to solve it via the type system alone isn't possible**. For instance, type-erased deleters (in `shared_ptr`) make it impossible to know if, given a `unique_ptr<X, D>` and a `shared_ptr<X>` object, a comparison between them is meaningless or instead they have somehow been designed to "work together".

A possible conservative solution could be to ban the mixed comparison via an explicit constraint/requirement on the proposed operators. That would however be nothing but a band-aid measure, as it wouldn't extend to other owning and non-owning smart pointer classes not from the Standard Library (like Boost's, Qt's, and so on).

In conclusion: given that

1. although admittely rarely used in practice, the operation is still meaningful;
2. a proper solution is not implementable in the type system; and
3. simply disallowing two smart pointer classes (from the Standard Library) while allowing third-party ones creates a major inconsistency,

in the present proposal **we are not going to explicitly ban the comparisons between different smart pointer classes**.

Despite this fact, the **current version of the proposal still forbids them**, although through indirect means: the § 6.2 Proposed wording disallows them due to the requirement clauses, which are currently unsatisfied (see § 5.3 Would these operations make the equality_comparable_with or three_way_comparable_with concepts satisfied between a smart pointer and a raw pointer?) when using the Standard Library smart pointer classes.

Other owning and non-owning smart pointer classes, not from the Standard Library, may or may not end up being comparable to the ones in the Standard Library using the operators that we are proposing, depending on whether they satisfy or not the requirements.

> A noteworthy case is `boost::intrusive_ptr<T>`, which satisfies them -- also because, notably, it features an implicit conversion from `T*`. Thanks to Peter Dimov for pointing it out.

§ 5.5. Should the proposed operators be `constexpr`?

If [P2273] gets adopted, then the proposed operators for `unique_ptr` should indeed become `constexpr`. On the other hand, the ones for `shared_ptr` shouldn't, for consistency with the pre-existing ones.

§ 6. Technical Specifications

All the proposed changes are relative to [N4892].

§ 6.1. Feature testing macro

Add to the list in [version.syn]:

```
#define __cpp_lib_mixed_smart_pointer_comparisons YYYYMML  // also in <memory>
```

with the value specified as usual (year and month of adoption).

6.2. Proposed wording

6.2.1. `unique_ptr`

Modify [unique.ptr.single.general] as shown:

```
    // [unique.ptr.single.mixed.cmp], mixed comparisons
    template<class U>
      requires equality_comparable_with<pointer, U>
      friend bool
      operator==(const unique_ptr& x, const U& y);
    template<class U>
      requires three_way_comparable_with<pointer, U>
      friend compare_three_way_result_t<pointer, U>
      operator<=>(const unique_ptr& x, const U& y);


    // disable copy from lvalue
    unique_ptr(const unique_ptr&) = delete;
    unique_ptr& operator=(const unique_ptr&) = delete;
};
```

Add a new section at the end of the [unique.ptr.single] chapter:

> **?.?.?.?.? Mixed comparison operators** [unique.ptr.single.mixed.cmp]
>
> ```
> template<class U>
>   requires equality_comparable_with<pointer, U>
>   friend bool
>     operator==(const unique_ptr& x, const U& y);
> ```
>
> 1 *Constraints*: `U` is not derived from a specialization of `unique_ptr`, and `is_null_pointer_v<U>` is `false`.
>
> 2 *Returns*: `x.get() == y`.
>
> ```
> template<class U>
>   requires three_way_comparable_with<pointer, U>
>   friend compare_three_way_result_t<pointer, U>
>     operator<=>(const unique_ptr& x, const U& y);
> ```
>
> 3 *Constraints*: `U` is not derived from a specialization of `unique_ptr`, and `is_null_pointer_v<U>` is `false`.
>
> 4 *Returns*: `compare_three_way()(x.get(), y)`.

Modify [unique.ptr.runtime.general] as shown:

```
    // mixed comparisons
    template<class U>
      requires equality_comparable_with<pointer, U>
      friend bool
      operator==(const unique_ptr& x, const U& y);
    template<class U>
      requires three_way_comparable_with<pointer, U>
      friend compare_three_way_result_t<pointer, U>
      operator<=>(const unique_ptr& x, const U& y);


    // disable copy from lvalue
    unique_ptr(const unique_ptr&) = delete;
    unique_ptr& operator=(const unique_ptr&) = delete;
};
```

6.2.2. `shared_ptr`

Modify [util.smartptr.shared.general] as shown:

```
    template<class U>
      bool owner_before(const weak_ptr<U>& b) const noexcept;


    // [util.smartptr.shared.mixed.cmp], mixed comparisons
    template<class U>
      requires equality_comparable_with<element_type*, U>
      friend bool operator==(const shared_ptr& a, const U& b);

    template<class U>
      requires three_way_comparable_with<element_type*, U>
      friend compare_three_way_result_t<element_type*, U>
        operator<=>(const shared_ptr& a, const U& b);

};
```

Insert a new section after [util.smartptr.shared.cmp]:

> **?.?.?.? Mixed comparison operators** [util.smartptr.shared.mixed.cmp]
>
> ```
> template<class U>
>   requires equality_comparable_with<element_type*, U>
>   friend bool operator==(const shared_ptr& a, const U& b);
> ```
>
> 1 *Constraints*: `U` is not derived from a specialization of `shared_ptr`, and `is_null_pointer_v<U>` is `false`.
> 2 *Returns*: `a.get() == b`.
>
> ```
> template<class U>
>   requires three_way_comparable_with<element_type*, U>
>   friend compare_three_way_result_t<element_type*, U>
>     operator<=>(const shared_ptr& a, const U& b);
> ```
>
> 3 *Constraints*: `U` is not derived from a specialization of `shared_ptr`, and `is_null_pointer_v<U>` is `false`.
> 4 *Returns*: `compare_three_way()(a.get(), b)`.

7. Implementation experience

A working prototype of the changes proposed by this paper, done on top of GCC 11, is available in this GCC branch on GitHub.

8. Acknowledgements

Credits for this idea go to Marc Mutz, who raised the question on the LEWG reflector, receiving a positive feedback.

Thanks to the reviewers of early drafts of this paper on the std-proposals mailing list.

Thanks to KDAB for supporting this work.

All remaining errors are ours and ours only.

References

Informative References

[MarcMutzReflector]
Marc Mutz. *unique_ptr<T> @ T* relational operators / comparison*. URL: https://lists.isocpp.org/lib-ext/2020/07/15873.php

[Meyers]
Scott Meyers. *Effective Modern C++, Chapter 4. Smart Pointers*. URL: https://www.oreilly.com/library/view/effective-modern-c/9781491908419/ch04.html

[N4892]
Thomas Köppe. *Working Draft, Standard for Programming Language C++*. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/n4892.pdf

[P0805R2]
Marshall Clow. *Comparing Containers*. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0805r2.html

[P0919R3]
Mateusz Pusz. *Heterogeneous lookup for unordered containers*. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0919r3.html

[P1614R2]
Barry Revzin. *The Mothership has Landed*. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1614r2.html

[P2249-GCC]
Giuseppe D'Angelo. *P2249 prototype implementation*. URL: https://github.com/dangelog/gcc/tree/std-proposals

[P2273]
Andreas Fertig. *Making std::unique_ptr constexpr*. URL: https://wg21.link/P2273

[P2403]
Justin Bassett. *nullopt_t and nullptr_t should both have operator<=> and operator==*. URL: https://wg21.link/P2403

[P2404]
Justin Bassett. *Relaxing comparison_relation_with's common reference requirements to support move-only types*. URL: https://wg21.link/P2404

[R.20]
Bjarne Stroustrup; Herb Sutter. *C++ Core Guidelines, R.20: Use `unique_ptr` or `shared_ptr` to represent ownership*. URL: https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#r20-use-unique_ptr-or-shared_ptr-to-represent-ownership

[SO_unique_ptr_comparable_thread]
Why is unique_ptr not equality_comparable_with nullptr_t in C++20?. URL: https://stackoverflow.com/questions/66937947/why-is-unique-ptr-not-equality-comparable-with-nullptr-t-in-c20

[Std-proposals]
P2249 discussion on the std-proposals mailing list. URL: https://lists.isocpp.org/std-proposals/2021/01/2308.php

[Sutter]
Herb Sutter. *Elements of Modern C++ Style*. URL: https://herbsutter.com/elements-of-modern-c-style/

§ Issues Index

ISSUE 1 What does LEWG(I) think about this? ↵